

National University of Computer and Emerging Sciences

CS 2006 - Operating Systems

Chrono Rift

Project Report

Spring 2026

Group Member	Roll Number
Muhammad Mughees Tariq Khawaja	24i-0806
Muhammad Mustafa	24i-0525

Submitted to: Department of Computer Science

Deadline: 10 May 2026

Table of Contents

1. Project Overview
2. Game Architecture
3. Turnaround Time Analysis
4. Gameplay Screenshots
5. Conclusion

1. Project Overview

Chrono Rift is a turn-based tactical game built as a semester project for CS 2006 - Operating Systems. The game runs as a multi-process application where a human player's party fights against computer-controlled enemies across waves of combat. The project was designed to put key operating systems concepts into practice (processes, threads, shared memory, semaphores, signals, deadlock handling, and memory management) all working together to produce a playable, real-time game.

We built the project in C++ and used **SFML** for the graphical interface. The entire solution compiles and runs inside the provided Docker environment using a Makefile.

2. Game Architecture

2.1 Process Structure

We split the game into three separate processes, each with a distinct responsibility. The **Game Arbiter** is the central authority - it owns the global game state, enforces turn order, spawns the other two processes, and manages the full game loop. The **Human Interfacing Process (HIP)** handles all keyboard input from the player and forwards actions to the Arbiter through shared memory. The **Automated Strategic Process (ASP)** manages all enemy decision-making, with each enemy running in its own dedicated thread inside the ASP.

2.2 Shared Memory and IPC

All communication between processes goes through a single shared memory segment. No pipes are used anywhere in the project. The shared memory holds the complete game state in a **SharedState** structure - player and enemy data, turn queues, the action log, artifact ownership, and all control flags. Access to this structure is protected by unnamed POSIX semaphores stored directly inside the shared memory region.

2.3 Multithreading

Inside the Arbiter, we run several threads concurrently: a **Stamina Timer** thread that increments stamina every second, a **Deadlock Monitor** thread that continuously checks for circular waits among artifact holders, a **Renderer** thread that reads the shared state and updates the SFML window, and a **Game Loop** thread that drives turn selection and action resolution. Inside the ASP, every individual enemy gets its own dedicated thread which run concurrently and only one acts at a time when the Arbiter grants a turn.

2.4 Signals

We use POSIX signals for two key mechanics. The **Stun** mechanic delivers **SIGUSR1** to a target process, which causes it to pause for exactly 3 seconds before resuming from where it left off. The **Ultimate Ability** sends **SIGSTOP** to suspend the ASP entirely for 10 seconds; the Arbiter uses **SIGALRM** and a custom handler to resume it with **SIGCONT** once the window expires. Quit is handled by sending **SIGTERM** to the Arbiter from the HIP.

2.5 Deadlock Detection and Resolution

Artifacts (the Solar Core, Lunar Blade, and dynamically spawned Eclipse Relic) are exclusive global resources. We maintain a shared resource table tracking who holds each artifact and who is waiting for one. The Deadlock Monitor thread runs a depth-first cycle-detection

algorithm on this wait-for graph every second. If a circular wait is detected, the monitor forces the victim process to release its held artifact, breaking the deadlock so the game can continue.

2.6 Inventory Memory Management

Each player has a primary inventory modelled as a linear array of 20 contiguous slots. When a weapon is picked up, our **InventoryAllocator** searches for a contiguous free block large enough to hold it. If no block exists, it swaps out the minimum number of weapons to long-term storage to free up enough space. The Solar Core and Lunar Blade each occupy 10 slots, meaning holding both fills the inventory completely, a hard constraint enforced by the allocator. The Splinter Stick's 2-slot footprint is handled correctly even in fragmented conditions.

3. Turnaround Time Analysis

The scheduling model in Chrono Rift is stamina-based. Every second, each living and non-stunned entity has its **speed** value added to its current stamina. Once an entity's stamina reaches its maximum, it is enqueued in the ready queue and becomes eligible to act. After acting, its stamina resets to zero (or 50% on Skip). This is directly analogous to an arrival-time scheduling problem.

3.1 Arrival Time Formula

Given that stamina increments by speed each second and the entity starts from zero stamina at the beginning of a round, the number of seconds until first action is:

$$\text{Arrival Time (seconds)} = \text{ceil}(\text{Max Stamina} / \text{Speed})$$

After an action resets stamina to zero, the same formula applies for subsequent turns. If stamina resets to 50% (Skip action), the next arrival time is halved. Enemy stamina is paused entirely during an active Ultimate Ability.

3.2 Player Configuration (Roll Numbers: 24i-0806 and 24i-0525)

Player stats are derived from roll numbers. For this project, the combined roll number seed used is **245826** (as set in arbiter.cpp). Player speed = 100 / party size.

Party Size	Speed	Max Stamina	Arrival Time (secs)
1 Player	100	100	1
2 Players	50	100	2
3 Players	33	100	4
4 Players	25	100	4

3.3 Enemy Configuration

Enemy speed is a random value between 10 and 30, and their max stamina is 150. The table below shows arrival time for the fastest and slowest possible enemies.

Enemy Type	Speed	Max Stamina	Arrival Time (secs)
Slowest Enemy	10	150	15
Average Enemy	20	150	8
Fastest Enemy	30	150	5

3.4 Comparative Turnaround (4-Player Party vs Enemies)

With a 4-player party (speed = 25) and a fast enemy (speed = 30), the enemy acts before any player in the first round. With a 1-player party (speed = 100), the player reaches full stamina in 1 second - faster than any enemy. This trade-off is central to the party-size decision in the game.

Entity	Speed	Max Stamina	Rounds to 1st Action
Player (1 member)	100	100	1 sec
Player (2 members)	50	100	2 secs
Player (3 members)	33	100	4 secs
Player (4 members)	25	100	4 secs
Enemy (slow, spd=10)	10	150	15 secs
Enemy (avg, spd=20)	20	150	8 secs
Enemy (fast, spd=30)	30	150	5 secs

3.5 Stun Impact on Turnaround

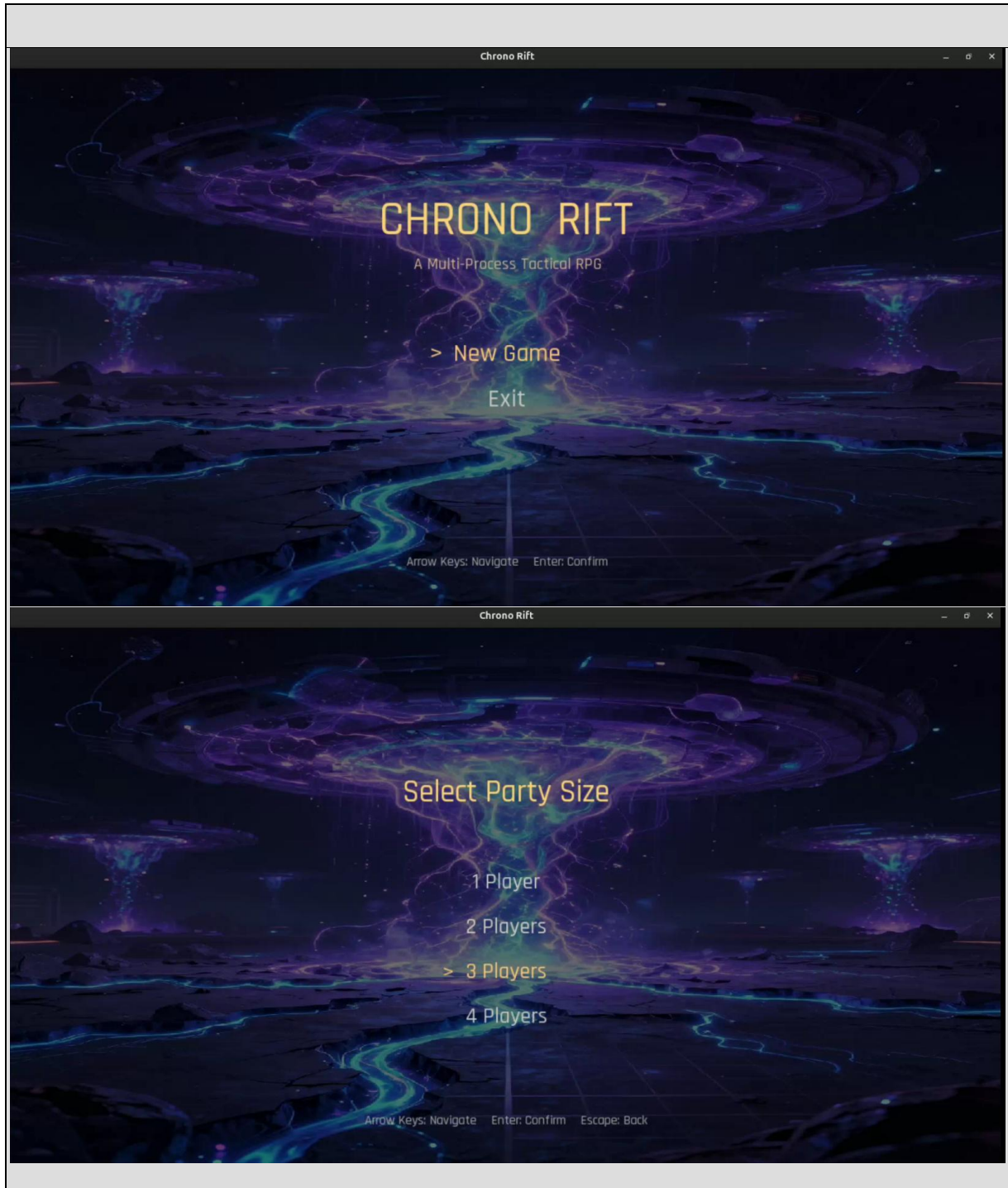
When a Stun is applied, the affected entity's stamina accumulation halts for exactly 3 seconds. If the entity's stamina was already full at the moment of the stun, its turn is skipped and it is only re-scheduled after the 3-second pause ends. This effectively adds 3 seconds to that entity's effective turnaround for that round. The stun is delivered asynchronously via **SIGUSR1** and handled in a signal handler - the target process does not poll for it.

3.6 Ultimate Ability Suspension

When the Ultimate Ability is triggered, the ASP (and therefore all enemy threads) is suspended via **SIGSTOP** for exactly 10 seconds. During this window, enemy stamina does not accumulate. This means all enemies effectively have their next arrival time pushed out by 10 seconds, or more, since the stamina they had at the moment of suspension is preserved when they resume.

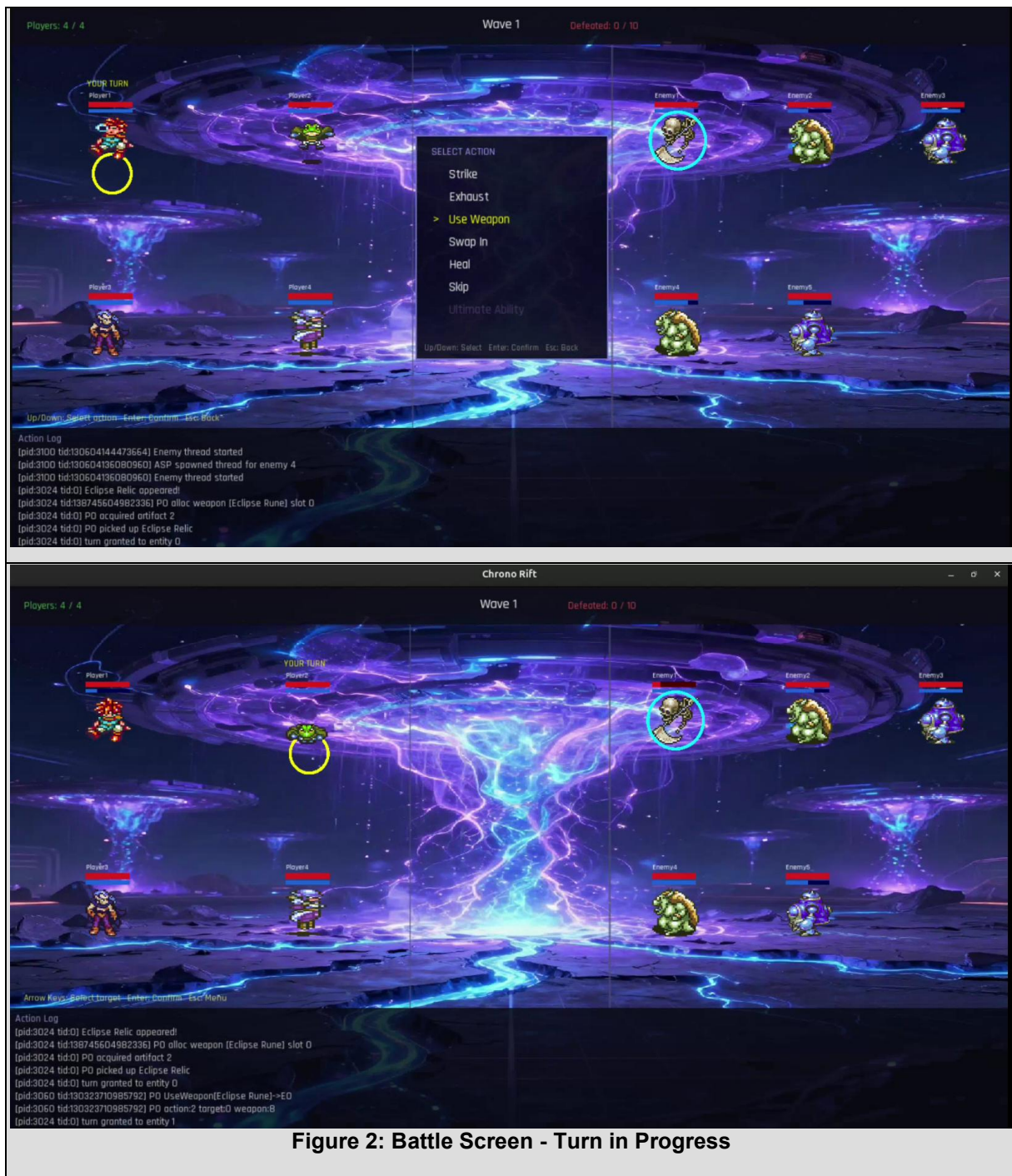
4. Gameplay Screenshots

Figure 1: Menu

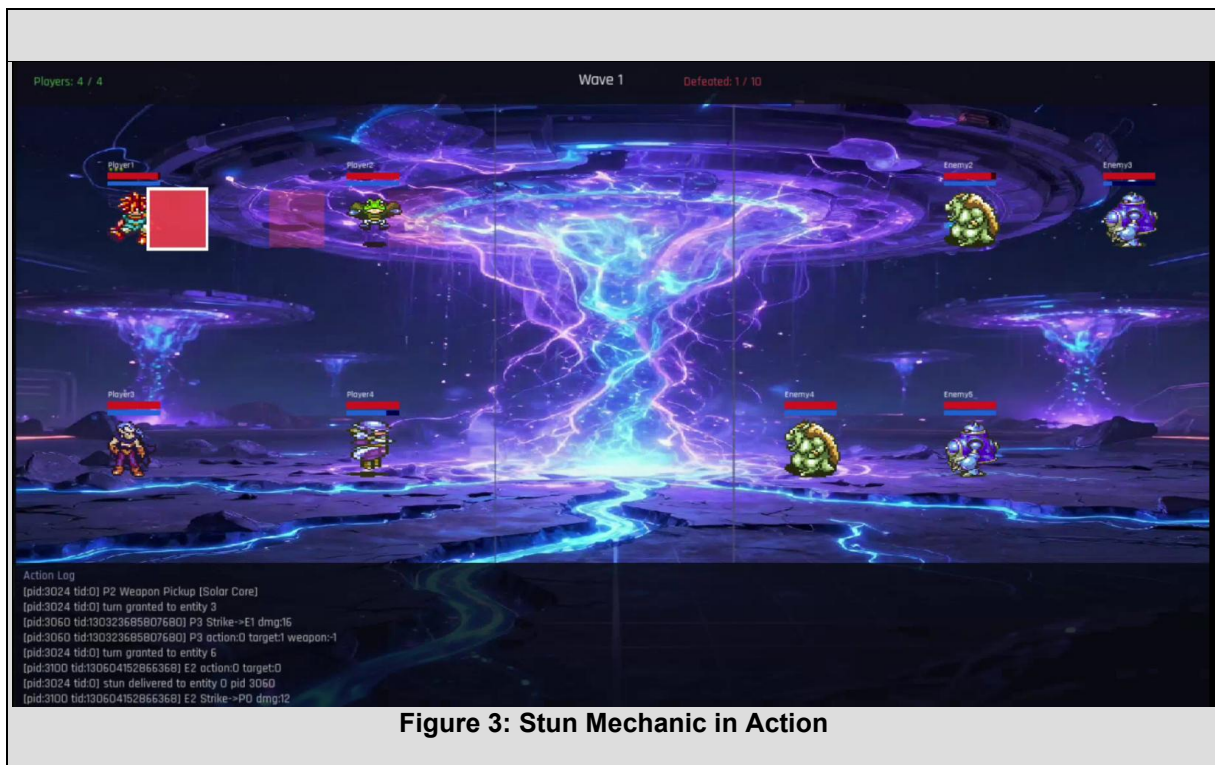


Main menu screen showing the game title, instructions, new game and exit game option.

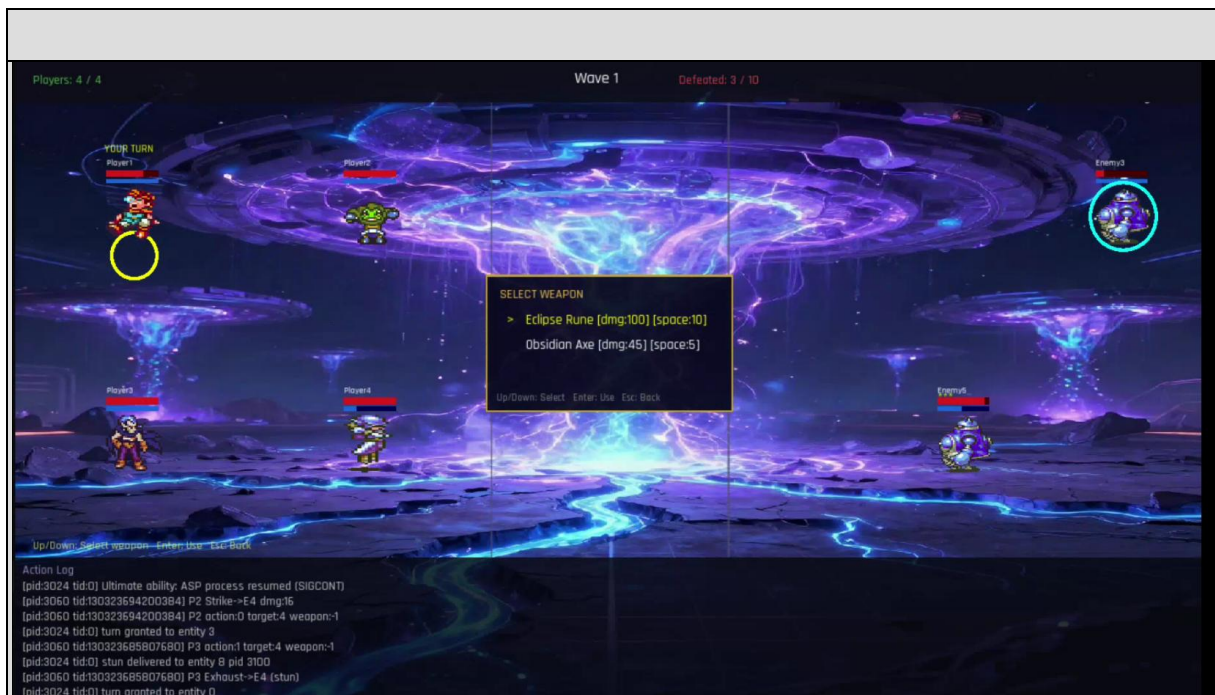
Select party screen showing party size selection with instructions.

Figure 2: Battle Screen - Turn in Progress**Figure 2: Battle Screen - Turn in Progress**

Active combat view with stamina bars, HP displays, action log, and player action menu visible.

Figure 3: Stun Mechanic in Action

An entity with the stun indicator(***) displayed above it after receiving a stun signal.

Figure 4: Inventory Screen

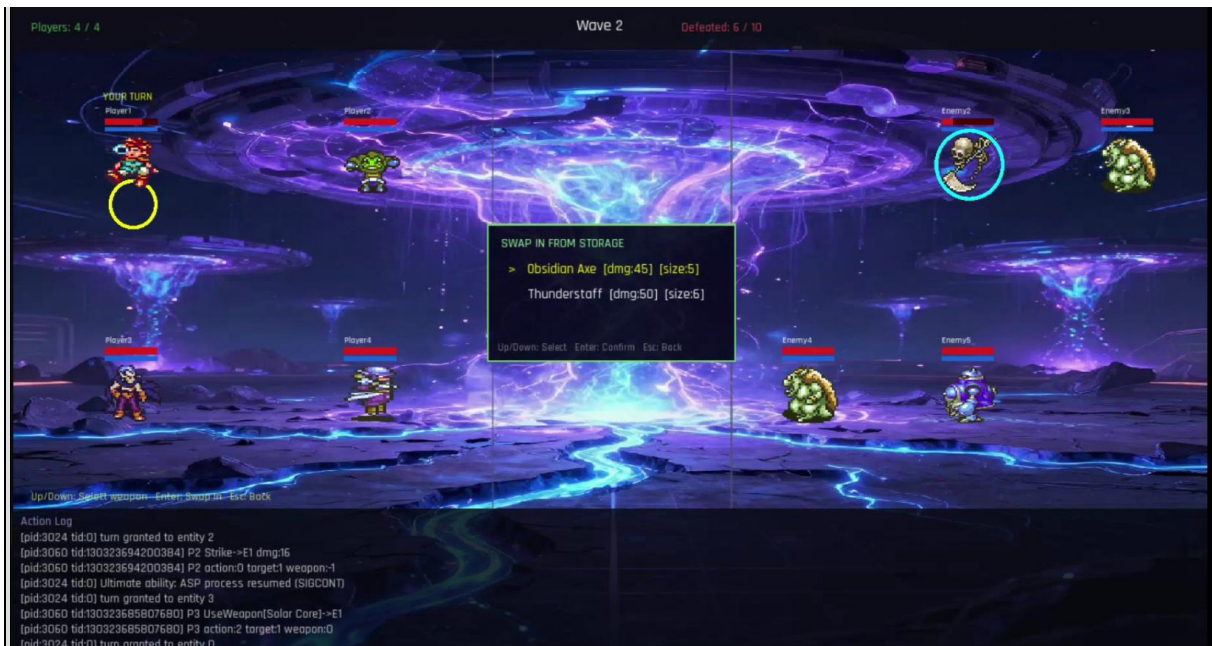
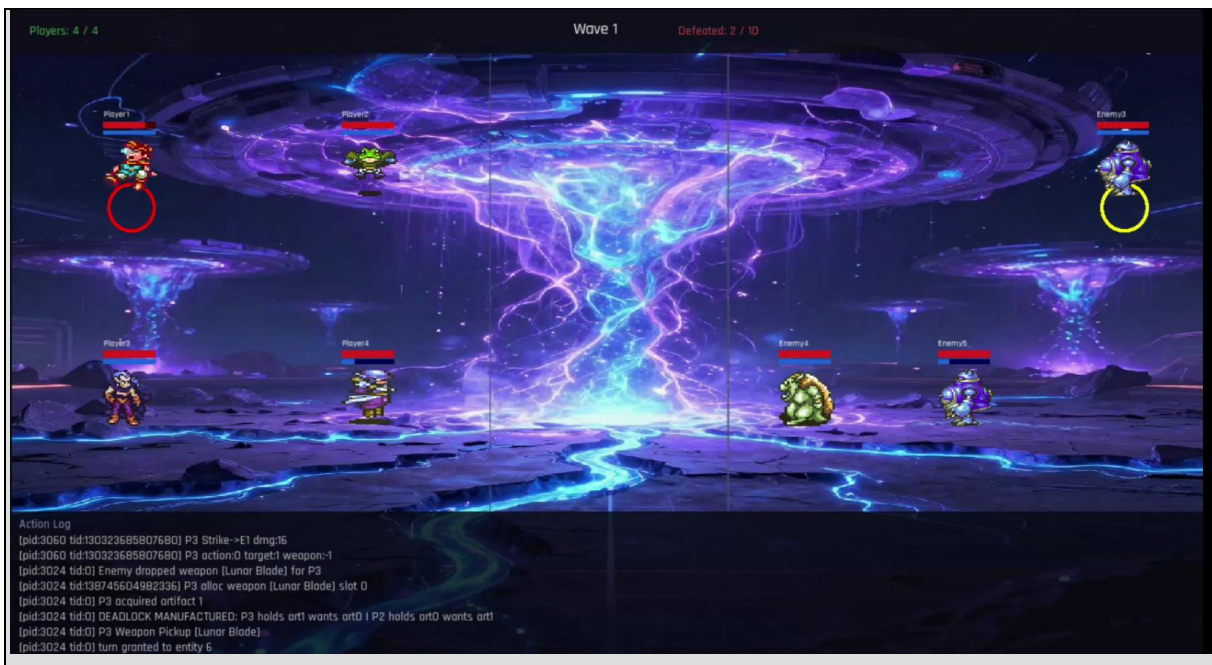


Figure 4: Inventory Screen

Player inventory showing contiguous slot allocation, held weapons, and long-term storage contents

Figure 5: Deadlock Detected - Resolution



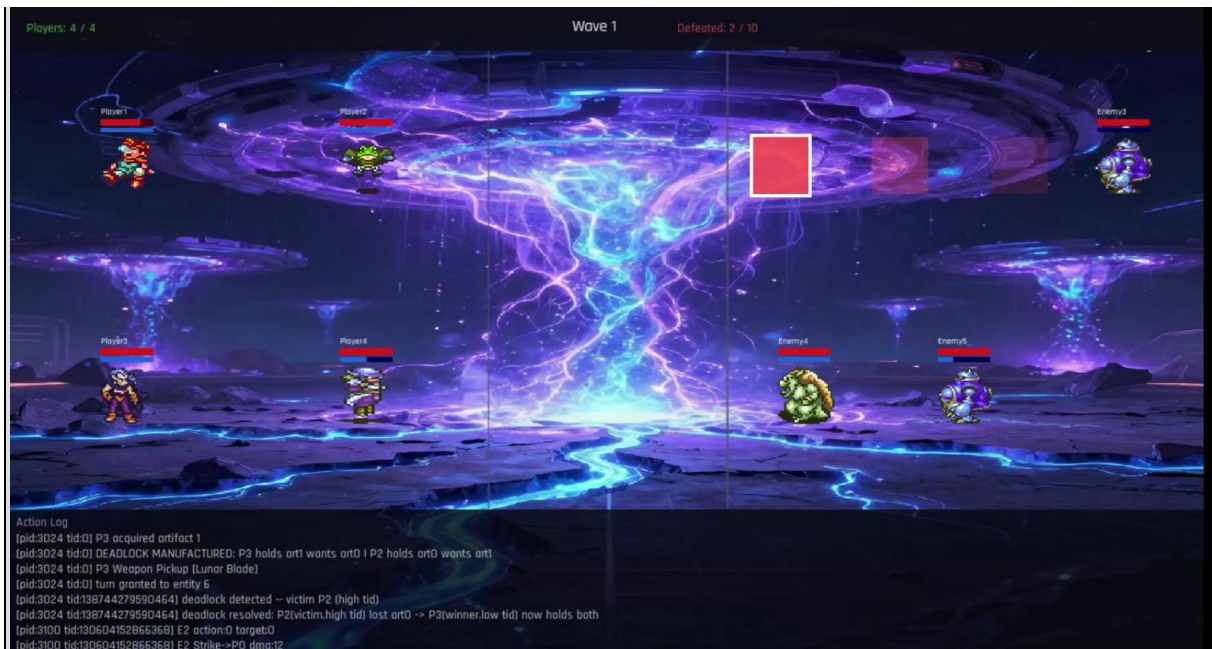
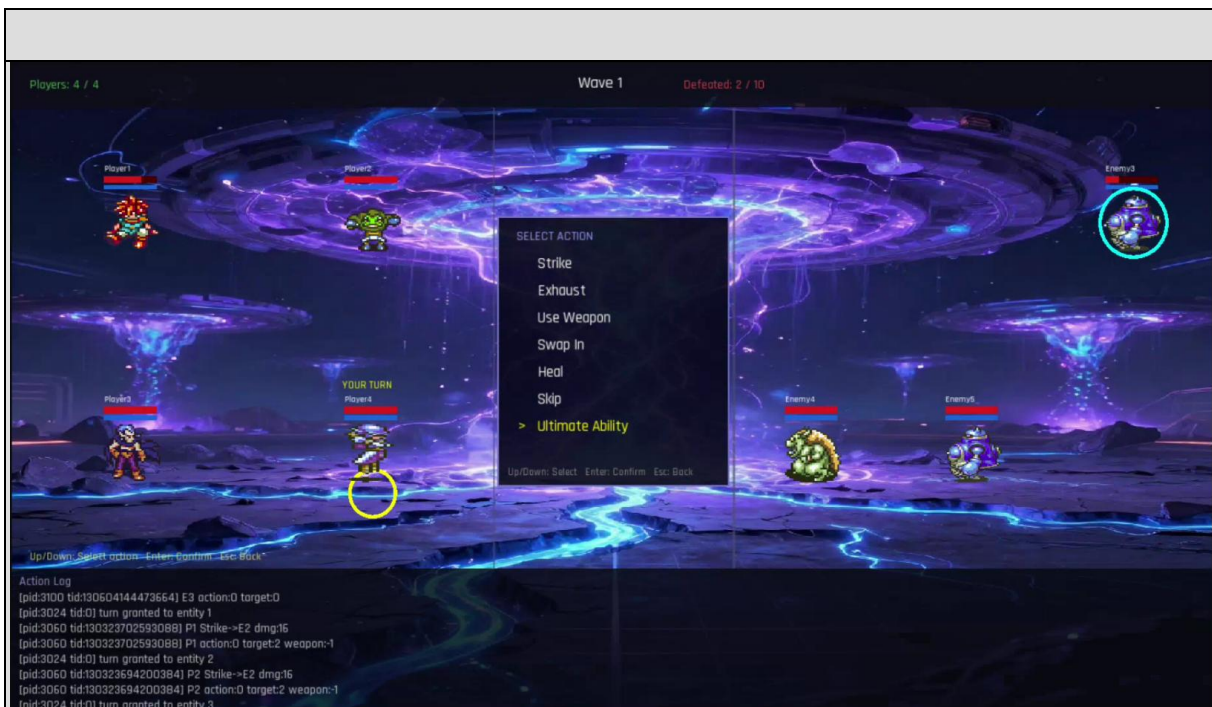


Figure 5: Deadlock Detected - Resolution

Action log entry showing deadlock detection and forced artifact release by the Deadlock Monitor thread.

Figure 6: Ultimate Ability Activated



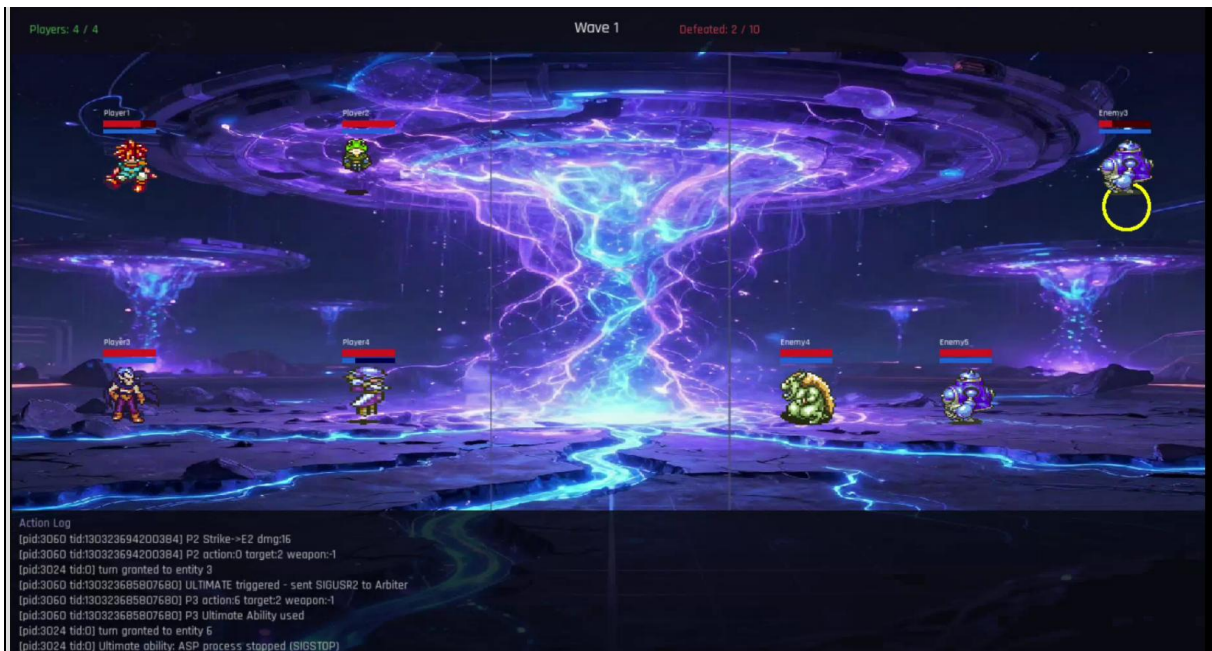
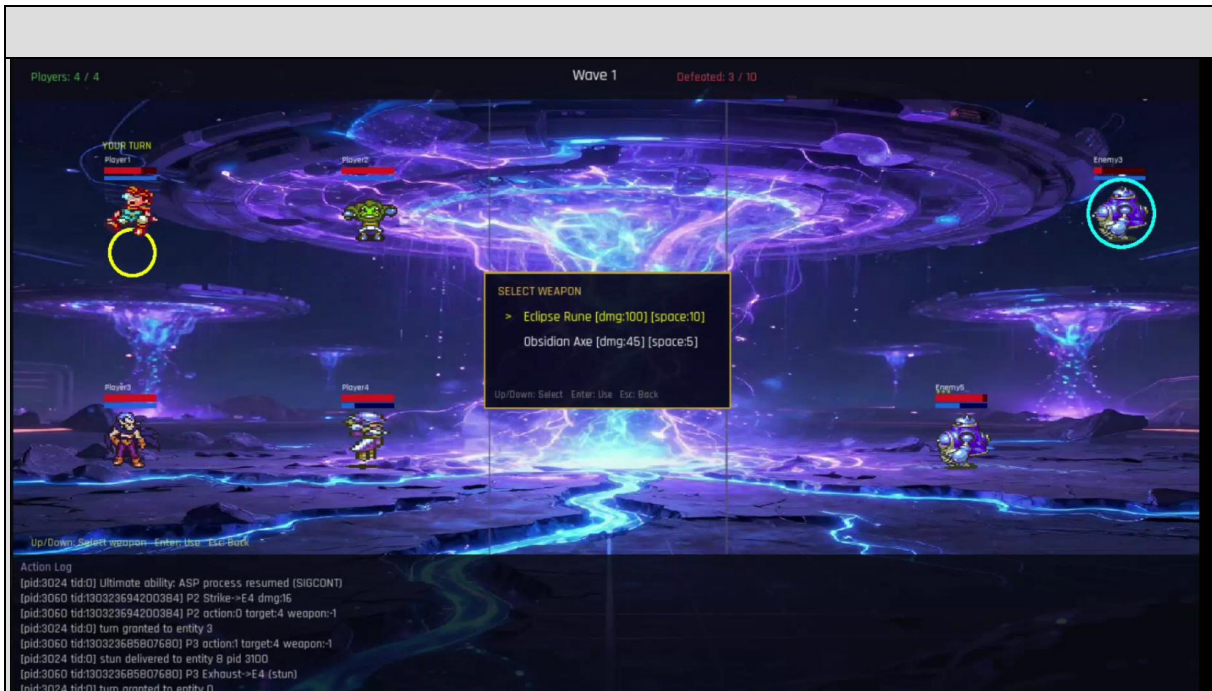


Figure 6: Ultimate Ability Activated

Screen state during a 10-second Ultimate window - enemy stamina bars frozen, player acting freely.

Figure 7: Wave Transition ,Victory Screen



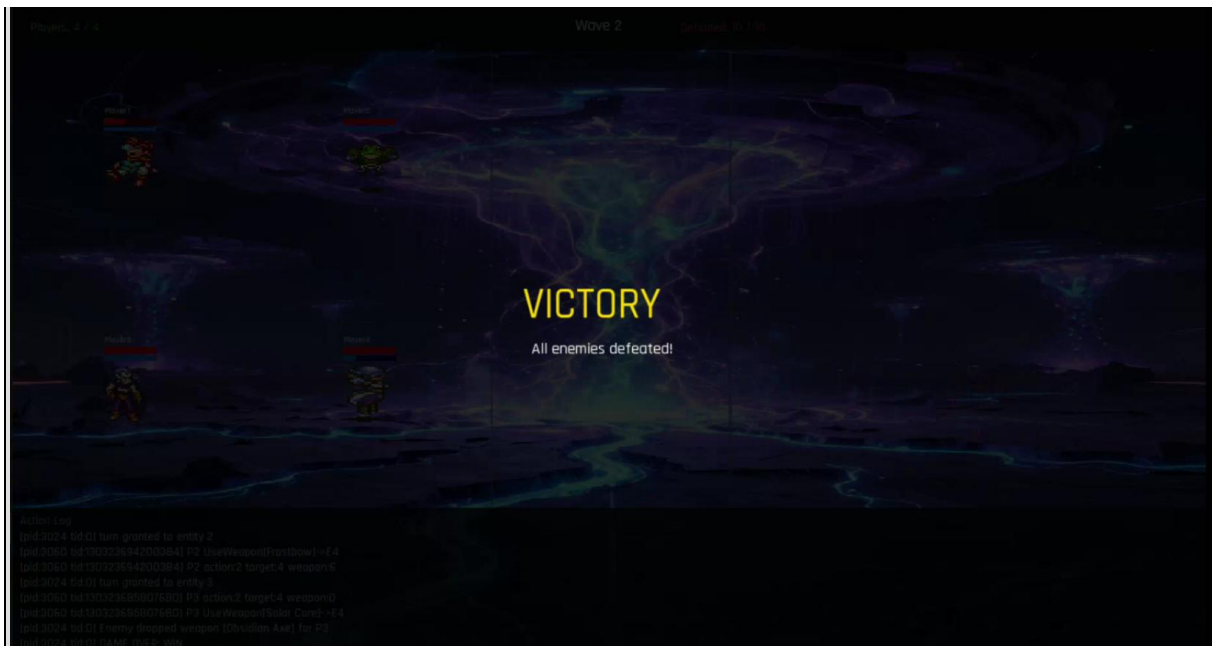


Figure 7: Wave Transition , Victory Screen

Wave completion or win condition screen displaying enemies defeated and result message.

5. Conclusion

Chrono Rift gave us hands-on experience with the core ideas taught in this course. Early in development, race conditions on the shared state caused the game to behave unpredictably. Fixing those issues by correctly using semaphores made a clear difference.

The stamina-based scheduler was one of the more interesting parts to get right. Mapping it to an arrival-time model helped us reason about when entities would act and why certain party configurations play out differently.

The signal-based stun and ultimate mechanics were tricky to implement correctly - especially making sure the stun handler preserved all state and that the ASP resumed from exactly the right point. The deadlock monitor was also challenging: we had to make sure the wait-for graph was always consistent with the actual artifact table, which required careful locking.

Overall, we are satisfied with the result. The game compiles cleanly in the Docker environment, all required OS features are implemented, and the SFML-based UI provides a clear real-time view of the underlying mechanics.